Boolean functions:

applications in quantum error correction and beyond

Bohan Lu

DQC Theory Seminar

PI: Ken Brown

September 16, 2026

Correcting one bit of error in memory



The *encoder* computes eight stored bits from four message bits. The *decoder* reconstructs the message from the values read back.

Assumption: at most one stored bit changes; the decoder does not know its position.

Extra stored values add constraints. They do not add new message information.

The encoder evaluates eight XOR expressions

XOR, written $\oplus$, returns 1 exactly when its bits differ: $0 \oplus 0 = 1 \oplus 1 = 0$, $0 \oplus 1 = 1 \oplus 0 = 1$.

Message: (b, a_1, a_2, a_3), with $a = (a_1, a_2, a_3)$. Address: $u = (u_1, u_2, u_3)$. Products of bits mean AND.

$$f_{a,b}(u) = b \oplus a_1 u_1 \oplus a_2 u_2 \oplus a_3 u_3.$$

u	000	001	010	011
$f_{a,b}(u)$	b	$b \oplus a_3$	$b \oplus a_2$	$b \oplus a_2 \oplus a_3$
example				

u	100	101	110	111
$f_{a,b}(u)$	$b \oplus a_1$	$b \oplus a_1 \oplus a_3$	$b \oplus a_1 \oplus a_2$	$b \oplus a_1 \oplus a_2 \oplus a_3$
example				

The encoder evaluates eight XOR expressions

XOR, written $\oplus$, returns 1 exactly when its bits differ: $0 \oplus 0 = 1 \oplus 1 = 0, \quad 0 \oplus 1 = 1 \oplus 0 = 1.$

Message: (b, a_1, a_2, a_3), with $a = (a_1, a_2, a_3)$. Address: $u = (u_1, u_2, u_3)$. Products of bits mean AND.

$$f_{a,b}(u) = b \oplus a_1 u_1 \oplus a_2 u_2 \oplus a_3 u_3.$$

u	000	001	010	011
$f_{a,b}(u)$	b	$b \oplus a_3$	$b \oplus a_2$	$b \oplus a_2 \oplus a_3$
example	0	1	1	0

u	100	101	110	111
$f_{a,b}(u)$	$b \oplus a_1$	$b \oplus a_1 \oplus a_3$	$b \oplus a_1 \oplus a_2$	$b \oplus a_1 \oplus a_2 \oplus a_3$
example	1	0	0	1

Example message $(0, 1, 1, 1)$ gives 01101001; at 101, $0 \oplus 1 \oplus 1 = 0$.

Affine Boolean functions define the codewords

A *Boolean function* assigns one bit to each binary address. We identify a function with its word of values in address order $000, 001, \ldots, 111$. Our encoder uses an *affine* function: a constant XOR selected input bits.

$$f_{a,b}(u) = b \oplus a_1 u_1 \oplus a_2 u_2 \oplus a_3 u_3.$$

Let e_i be the address with a 1 in position i and zeros elsewhere: $e_1 = 100$, $e_2 = 010$, $e_3 = 001$. An uncorrupted word gives back its message:

$$b = f_{a,b}(000), \qquad a_i = f_{a,b}(e_i) \oplus f_{a,b}(000).$$

Different messages therefore give different stored words. These 16 allowed words are the *codewords*; their collection is the *code* C.

C selects 16 words from the 256 possible eight-bit readouts.

From a truth table to a polynomial

Our example is parity, $p(u) = u_1 \oplus u_2 \oplus u_3$. The function r records the eight bits read from memory:

$$p \longleftrightarrow \texttt{01101001}, \qquad r \longleftrightarrow \texttt{1}1101001.$$

From a truth table to a polynomial

Our example is parity, $p(u) = u_1 \oplus u_2 \oplus u_3$. The function r records the eight bits read from memory:

$$p \longleftrightarrow \texttt{01101001}, \qquad r \longleftrightarrow \texttt{1}1101001.$$

The product $(1 \oplus u_1)(1 \oplus u_2)(1 \oplus u_3)$ equals 1 at 000 and 0 at every other address. Hence

$$r(u) = p(u) \oplus (1 \oplus u_1)(1 \oplus u_2)(1 \oplus u_3).$$

From a truth table to a polynomial

Our example is parity, $p(u) = u_1 \oplus u_2 \oplus u_3$. The function r records the eight bits read from memory:

$$p \longleftrightarrow 01101001, \qquad r \longleftrightarrow 11101001.$$

The product $(1 \oplus u_1)(1 \oplus u_2)(1 \oplus u_3)$ equals 1 at 000 and 0 at every other address. Hence

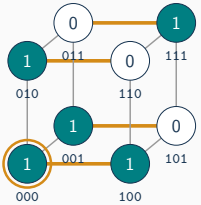
$$r(u) = p(u) \oplus (1 \oplus u_1)(1 \oplus u_2)(1 \oplus u_3).$$

Every Boolean function is a unique XOR of products of distinct input variables, its *algebraic normal form* (ANF). Expanding gives

$$r(u) = 1 \oplus u_1u_2 \oplus u_1u_3 \oplus u_2u_3 \oplus u_1u_2u_3.$$

The *degree* is the largest number of variables in a product term. Codewords have degree at most one. This readout has degree three, so it is not a codeword: an error occurred.

Comparing neighboring table entries

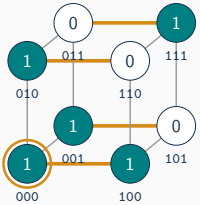


Values of r inside, addresses below. Teal: 1, white: 0. Gold marks first-bit edges and the changed entry.

Slide 3 uses the highlighted pair, which holds the changed entry:

$$p(000) \oplus p(100) = 0 \oplus 1 = 1,$$
$$r(000) \oplus r(100) = 1 \oplus 1 = 0 \quad \text{(wrong)}.$$

Comparing neighboring table entries



Values of r inside, addresses below. Teal: 1, white: 0. Gold marks first-bit edges and the changed entry.

Slide 3 uses the highlighted pair, which holds the changed entry:

$$p(000) \oplus p(100) = 0 \oplus 1 = 1,$$
$$r(000) \oplus r(100) = 1 \oplus 1 = 0 \quad \text{(wrong)}.$$

The *Boolean derivative* records this comparison:

$$D_{e_i} f(u) = f(u \oplus e_i) \oplus f(u).$$

Derivatives of p and r along $e_1 = 100$: four disjoint pairs.

edge	$000, 100$	$001, 101$	$010, 110$	$011, 111$
$D_{e_1} p$	1	1	1	1
$D_{e_1} r$	0	1	1	1

Recovering the four message coefficients

Toggling address bit i cancels every affine term except its coefficient:

$$D_{e_i} f_{a,b} = a_i.$$

The four disjoint first-direction edges give four votes for a_1:

addresses	$000, 100$	$001, 101$	$010, 110$	$011, 111$
received XOR	0	1	1	1

Why is this majority reliable?

Recovering the four message coefficients

Toggling address bit i cancels every affine term except its coefficient:

$$D_{e_i} f_{a,b} = a_i.$$

The four disjoint first-direction edges give four votes for a_1:

addresses	$000, 100$	$001, 101$	$010, 110$	$011, 111$
received XOR	0	1	1	1

One changed entry spoils at most one of four comparisons in each direction. Majority gives $a_1 = 1$, and directions e_2, e_3 give $a_2 = a_3 = 1$. Removing those terms leaves

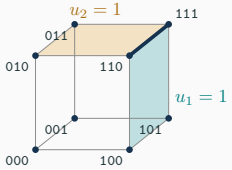
$$r(u) \oplus a_1 u_1 \oplus a_2 u_2 \oplus a_3 u_3 \longleftrightarrow 10000000.$$

Seven entries give $b = 0$: message $(0, 1, 1, 1)$.

Recovered: three coefficients and the constant.

Intersections count differing positions

Could a second codeword also lie one change from r? Count where two words differ. The *support* is the set of positions containing a one, and the *weight* $\mathrm{wt}(f)$ counts them.



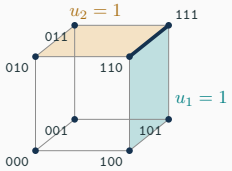
Labels are addresses. Colored faces mark ones; the dark edge is their intersection.

$$\mathrm{wt}(u_1) = 4, \quad \mathrm{wt}(u_2) = 4, \qquad \mathrm{wt}(u_1 u_2) = 2.$$

XOR keeps positions in exactly one support. The common edge was counted twice and must be removed twice.

Intersections count differing positions

Could a second codeword also lie one change from r? Count where two words differ. The *support* is the set of positions containing a one, and the *weight* $\mathrm{wt}(f)$ counts them.



Labels are addresses. Colored faces mark ones; the dark edge is their intersection.

$$\mathrm{wt}(u_1) = 4, \quad \mathrm{wt}(u_2) = 4, \qquad \mathrm{wt}(u_1 u_2) = 2.$$

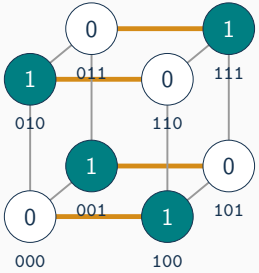
XOR keeps positions in exactly one support. The common edge was counted twice and must be removed twice.

$$\mathrm{wt}(f \oplus g) = \mathrm{wt}(f) + \mathrm{wt}(g) - 2\,\mathrm{wt}(fg).$$

Hamming distance counts positions where two words differ:

$$d_H(f, g) = \mathrm{wt}(f \oplus g), \qquad d_H(u_1, u_2) = 4 + 4 - 2(2) = 4.$$

Nonconstant affine functions have four ones



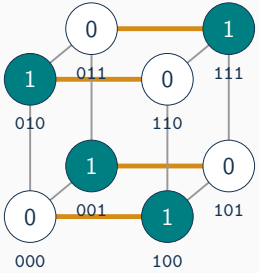
Parity: addresses below, values inside. Each gold edge contains one 0 and one 1.

Choose a coordinate that appears in the affine function. Toggling that coordinate changes the output.

4 disjoint pairs $\implies$ 4 ones.

Coordinate changes?

Nonconstant affine functions have four ones



Parity: addresses below, values inside. Each gold edge contains one 0 and one 1.

Choose a coordinate that appears in the affine function. Toggling that coordinate changes the output.

$$4 \text{ disjoint pairs} \implies 4 \text{ ones.}$$

The inputs giving a fixed output form an *affine hyperplane*. For parity, the four inputs giving 1 are

$$\{001, 010, 100, 111\}.$$

The parity-zero set passes through zero. The parity-one set is its translate. Slide 16 builds the 48-qubit code by deleting a linear subspace of addresses.

Coordinate changes?

Minimum distance makes correction unique

The *minimum distance* separates the closest distinct codewords:

$$d_{\min}(C) = \min_{\substack{f,g \in C \\ f \neq g}} d_H(f, g).$$

For distinct codewords f, g, the XOR $f \oplus g$ is a nonzero affine function (coefficients XOR). It has four ones if nonconstant, eight if constant. The pair u_1, u_2 attains four, so $d_{\min}(C) = 4$.

Minimum distance makes correction unique

The *minimum distance* separates the closest distinct codewords:

$$d_{\min}(C) = \min_{\substack{f,g \in C \\ f \neq g}} d_H(f, g).$$

For distinct codewords f, g, the XOR $f \oplus g$ is a nonzero affine function (coefficients XOR). It has four ones if nonconstant, eight if constant. The pair u_1, u_2 attains four, so $d_{\min}(C) = 4$.

codeword f $\xleftarrow{\text{at most 1 change}}$ readout r $\xleftarrow{\text{at most 1 change}}$ codeword g

Where f and g differ, r differs from at least one of them. Two different candidates would imply

$$4 \leq d_H(f, g) \leq d_H(f, r) + d_H(r, g) \leq 2.$$

One-error correction is unique.

What the classical example established

Affine evaluations	encode four bits as eight values
ANF	describes the corrupted table
Derivatives	recover the message coefficients
Minimum distance	proves the correction is unique

$$\underbrace{d_H(p, r) = 1}_{\text{one particular corruption}} \quad , \qquad \underbrace{d_{\min}(C) = 4}_{\text{separation of allowed words}} \quad .$$

The sixteen codewords form eight complementary pairs $f_{a,0}$, $f_{a,1} = f_{a,0} \oplus 11111111$. Each pair will represent one encoded quantum basis state.

A quantum state has amplitudes and relative phase

A qubit uses two basis vectors, written $|0\rangle = (1,0)^\mathsf{T}$ and $|1\rangle = (0,1)^\mathsf{T}$. A state is $\alpha|0\rangle + \beta|1\rangle$ with complex *amplitudes* α, β and $|\alpha|^2 + |\beta|^2 = 1$.

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}}$$ is a superposition of the two basis states.

A quantum state has amplitudes and relative phase

A qubit uses two basis vectors, written $|0\rangle = (1,0)^{\mathsf{T}}$ and $|1\rangle = (0,1)^{\mathsf{T}}$. A state is $\alpha|0\rangle + \beta|1\rangle$ with complex *amplitudes* α, β and $|\alpha|^2 + |\beta|^2 = 1$.

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad \text{is a superposition of the two basis states.}$$

Change one relative sign

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} \longmapsto \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

This phase flip changes the state.

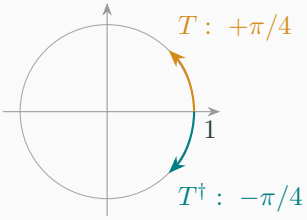
Change the whole state's sign

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} \longmapsto -\frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

An overall phase leaves the physical state unchanged.

The two basic errors: a *phase flip* changes the relative sign, a *bit flip* exchanges $|0\rangle$ and $|1\rangle$.

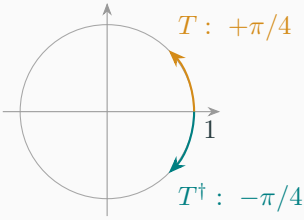
Single-qubit rotations add phase angles



Here $i^2 = -1$, and $e^{i\theta}$ is the unit complex number at angle θ. The rotation T leaves $|0\rangle$ fixed and multiplies the $|1\rangle$ amplitude by $e^{i\pi/4}$. $T^\dagger$ rotates in the opposite direction.

Goal: build a three-qubit logical gate from these single-qubit rotations alone. A rotation touches one physical qubit, so a fault in it stays on one qubit.

Single-qubit rotations add phase angles



Here $i^2 = -1$, and $e^{i\theta}$ is the unit complex number at angle θ. The rotation T leaves $|0\rangle$ fixed and multiplies the $|1\rangle$ amplitude by $e^{i\pi/4}$. $T^\dagger$ rotates in the opposite direction.

Goal: build a three-qubit logical gate from these single-qubit rotations alone. A rotation touches one physical qubit, so a fault in it stays on one qubit.

For a binary word v, $|v\rangle$ denotes the state with those physical bits. Applying T at every position multiplies one factor for each one:

$$|00101\rangle \longmapsto e^{2i\pi/4}|00101\rangle, \qquad |v\rangle \longmapsto e^{i\pi\,\mathrm{wt}(v)/4}|v\rangle.$$

Complementary words encode three qubits

An encoded (*logical*) qubit is stored across several physical qubits. Use the three-bit coefficient vector a as a *logical label*. Pair the affine words with constants $b = 0$ and $b = 1$:

a	$f_{a,0}$	$f_{a,1}$
000	00000000	11111111
100	00001111	11110000
111	01101001	10010110

$$|a\rangle_L = \frac{|f_{a,0}\rangle + |f_{a,1}\rangle}{\sqrt{2}}.$$

Complementary words encode three qubits

An encoded (*logical*) qubit is stored across several physical qubits. Use the three-bit coefficient vector a as a *logical label*. Pair the affine words with constants $b = 0$ and $b = 1$:

a	$f_{a,0}$	$f_{a,1}$
000	00000000	11111111
100	00001111	11110000
111	01101001	10010110

$$|a\rangle_L = \frac{|f_{a,0}\rangle + |f_{a,1}\rangle}{\sqrt{2}}.$$

Sums over disjoint sets of words are orthogonal. The eight pairs are disjoint, so they give $8 = 2^3$ orthogonal encoded basis states: **the eight-qubit code stores three logical qubits**.

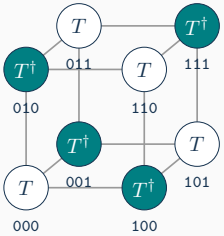
The **same phase on both words** preserves the encoded basis state up to an overall phase.

Choosing rotations for a three-qubit sign change

Target gate: **CCZ** (controlled-controlled-Z) gives only logical label 111 a minus sign:

$$|a\rangle_L \longmapsto (-1)^{a_1 a_2 a_3} |a\rangle_L.$$

CCZ is not a Clifford gate, and adding it to the Clifford gates gives universal quantum computation.



Address below each vertex. White: parity 0, apply T. Teal: parity 1, apply $T^\dagger$.

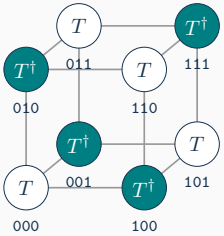
Try the pattern given by parity p: T where $p = 0$, $T^\dagger$ where $p = 1$. For $v = p$, all four ones lie at $T^\dagger$ sites: the total angle is $-4\pi/4$.

Choosing rotations for a three-qubit sign change

Target gate: **CCZ** (controlled-controlled-Z) gives only logical label 111 a minus sign:

$$|a\rangle_L \longmapsto (-1)^{a_1 a_2 a_3} |a\rangle_L.$$

CCZ is not a Clifford gate, and adding it to the Clifford gates gives universal quantum computation.



Address below each vertex. White: parity 0, apply T. Teal: parity 1, apply $T^\dagger$.

Try the pattern given by parity p: T where $p = 0$, $T^\dagger$ where $p = 1$. For $v = p$, all four ones lie at $T^\dagger$ sites: the total angle is $-4\pi/4$.

The *signed weight* counts, for a physical word v,

$$W(v) = (\text{number of ones at } T) - (\text{number of ones at } T^\dagger).$$

The resulting physical-word phase is $e^{i\pi W(v)/4}$.

Checking every encoded pair

Site x contributes $+v(x)$ to $W(v)$ if $p(x) = 0$ and $-v(x)$ if $p(x) = 1$. Both cases equal the integer $(p \oplus v)(x) - p(x)$. Summing over sites, with $\mathrm{wt}(p) = 4$,

$$W(v) = \mathrm{wt}(p \oplus v) - 4 = d_H(p, v) - 4.$$

Since $p \oplus f_{a,b} = f_{a \oplus 111, b}$, for $a \neq 111$ it is nonconstant affine and has weight four.

logical label	$W(f_{a,0})$	$W(f_{a,1})$	phase
$a \neq 111$	0	0	
$a = 111$	-4	$+4$	

Do $+4$ and -4 give different phases when each count contributes an angle of $\pi/4$?

Checking every encoded pair

Site x contributes $+v(x)$ to $W(v)$ if $p(x) = 0$ and $-v(x)$ if $p(x) = 1$. Both cases equal the integer $(p \oplus v)(x) - p(x)$. Summing over sites, with $\mathrm{wt}(p) = 4$,

$$W(v) = \mathrm{wt}(p \oplus v) - 4 = d_H(p, v) - 4.$$

Since $p \oplus f_{a,b} = f_{a \oplus 111, b}$, for $a \neq 111$ it is nonconstant affine and has weight four.

logical label	$W(f_{a,0})$	$W(f_{a,1})$	phase
$a \neq 111$	0	0	$+1$
$a = 111$	-4	$+4$	-1

$e^{i\pi} = e^{-i\pi} = -1$**.** Both words in each pair receive the same phase. Exactly logical label 111 changes sign: the physical rotations implement CCZ.

Selecting 48 physical positions

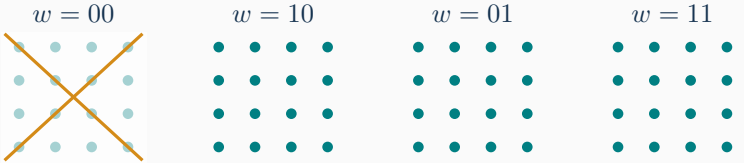
The eight-qubit code does not correct every one-qubit error. For the protected construction, use **six-bit addresses**: $u = (u_1, u_2, u_3, u_4)$ within a block (now four bits, previously three) and $w = (w_1, w_2)$ labeling the block.



Why is the eight-qubit code insufficient?

Selecting 48 physical positions

The eight-qubit code does not correct every one-qubit error. For the protected construction, use **six-bit addresses**: $u = (u_1, u_2, u_3, u_4)$ within a block (now four bits, previously three) and $w = (w_1, w_2)$ labeling the block.



Delete the block $w = 00$. Each of the $64 - 16 = 48$ retained addresses is one physical qubit. The deletion makes every nonzero linear weight divisible by eight (next slide), which the gate test needs.

Why is the eight-qubit code insufficient?

The retained linear words have weights 24 and 32

A linear function is an XOR of selected address bits, with no constant term. Every nonzero linear function has **32 ones on 64 addresses**: use the earlier pairing argument.

example	full space	deleted $w = 00$	retained

The retained linear words have weights 24 and 32

A linear function is an XOR of selected address bits, with no constant term. Every nonzero linear function has **32 ones on 64 addresses**: use the earlier pairing argument.

example	full space	deleted $w = 00$	retained
u_1	32	8	$32 - 8 = 24$

If any u bit occurs, the function still splits the deleted block into two equal halves.

The retained linear words have weights 24 and 32

A linear function is an XOR of selected address bits, with no constant term. Every nonzero linear function has **32 ones on 64 addresses**: use the earlier pairing argument.

example	full space	deleted $w = 00$	retained
u_1	32	8	$32 - 8 = 24$
w_1	32	0	$32 - 0 = 32$

If any u bit occurs, the function still splits the deleted block into two equal halves.

Otherwise only w bits occur, and the function is zero throughout the deleted block.

Every nonzero retained weight is 24 or 32: never zero, and divisible by eight. Slide 18 uses the first fact, slide 22 a signed version of the second.

A sum over 64 words is unchanged by bit flips

Evaluate the six coordinate functions $u_1, u_2, u_3, u_4, w_1, w_2$ on the 48 sites. Call the resulting words $S_1, \ldots, S_6$. All their XOR combinations, including the empty one $0 \cdots 0$, form a collection S. No nonzero combination vanishes (weight 24 or 32), so S has $2^6 = 64$ words.

$$\frac{1}{8} \sum_{v \in S} |v\rangle, \qquad \text{each amplitude } 1/8 : \; 64(1/8)^2 = 1.$$

The eight-qubit code is the case $S = \{00000000, 11111111\}$ with amplitude $1/\sqrt{2}$.

A sum over 64 words is unchanged by bit flips

Evaluate the six coordinate functions $u_1, u_2, u_3, u_4, w_1, w_2$ on the 48 sites. Call the resulting words $S_1, \ldots, S_6$. All their XOR combinations, including the empty one $0 \cdots 0$, form a collection S. No nonzero combination vanishes (weight 24 or 32), so S has $2^6 = 64$ words.

$$\frac{1}{8} \sum_{v \in S} |v\rangle, \qquad \text{each amplitude } 1/8 : \; 64(1/8)^2 = 1.$$

The eight-qubit code is the case $S = \{00000000, 11111111\}$ with amplitude $1/\sqrt{2}$. Flipping the physical bits selected by S_1 pairs up the summands:

$$|0 \cdots 0\rangle \;\longleftrightarrow\; |S_1\rangle, \qquad |S_2\rangle \;\longleftrightarrow\; |S_1 \oplus S_2\rangle.$$

The sum is unchanged. The same holds for each S_i. Operations that leave every encoded state unchanged are *stabilizers* of the encoding.

Logical functions distinguish eight encoded states

Three Boolean functions (backup slide) give 48-bit words K_1, K_2, K_3. For logical label $a = (a_1, a_2, a_3)$, define

$$K(a) = a_1 K_1 \oplus a_2 K_2 \oplus a_3 K_3.$$

In the eight-qubit code, $K_i = u_i$ and $K(a) = f_{a,0}$. For label 100, add K_1 to each word in S:

$$\{0 \cdots 0, S_1, S_2, \ldots\} \longmapsto \{K_1, K_1 \oplus S_1, K_1 \oplus S_2, \ldots\}.$$

What are the logical functions?

Logical functions distinguish eight encoded states

Three Boolean functions (backup slide) give 48-bit words K_1, K_2, K_3. For logical label $a = (a_1, a_2, a_3)$, define

$$K(a) = a_1 K_1 \oplus a_2 K_2 \oplus a_3 K_3.$$

In the eight-qubit code, $K_i = u_i$ and $K(a) = f_{a,0}$. For label 100, add K_1 to each word in S:

$$\{0 \cdots 0, S_1, S_2, \ldots\} \longmapsto \{K_1, K_1 \oplus S_1, K_1 \oplus S_2, \ldots\}.$$

The chosen logical words give eight disjoint collections, hence eight orthogonal states:

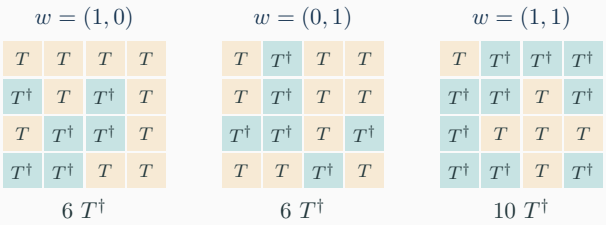
$$|a\rangle_L = \frac{1}{8} \sum_{v \in S} |K(a) \oplus v\rangle.$$

Adding any S_i still permutes the 64 terms. Those operations leave every encoded state fixed.

What are the logical functions?

Assigning the physical rotations

Fix this physical pattern. Gold sites receive T; teal sites receive $T^\dagger$.



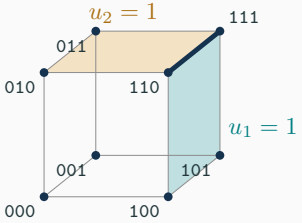
Within each block: rows u_1u_2, columns u_3u_4, both ordered $00, 01, 11, 10$.

There are 26 T and 22 $T^\dagger$. This pattern plays the role parity played on slide 14. For this fixed pattern, $W(v)$ counts ones at T positively and at $T^\dagger$ negatively.

How is this pattern specified?

XOR produces intersection terms

Each encoded word $K(a) \oplus v$ is an XOR of the nine *generator rows* $S_1, \ldots, S_6, K_1, K_2, K_3$. W is a signed sum of entries, so it splits over integer $+$ but not over $\oplus$. For distinct rows g_1, g_2, g_3, rewrite XOR with $+$, $-$, and products (coordinatewise AND).



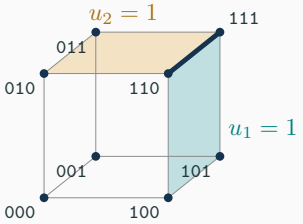
Recall the two coordinate supports and their common edge.

At every position, as an ordinary integer identity,

$$g_1 \oplus g_2 = g_1 + g_2 - 2g_1g_2.$$

XOR produces intersection terms

Each encoded word $K(a) \oplus v$ is an XOR of the nine *generator rows* $S_1, \ldots, S_6, K_1, K_2, K_3$. W is a signed sum of entries, so it splits over integer $+$ but not over $\oplus$. For distinct rows g_1, g_2, g_3, rewrite XOR with $+$, $-$, and products (coordinatewise AND).



Recall the two coordinate supports and their common edge.

At every position, as an ordinary integer identity,

$$g_1 \oplus g_2 = g_1 + g_2 - 2g_1g_2.$$

For three rows,

$$\begin{aligned} g_1 \oplus g_2 \oplus g_3 = g_1 + g_2 + g_3 \\ - 2(g_1g_2 + g_1g_3 + g_2g_3) \\ + 4g_1g_2g_3. \end{aligned}$$

Count individual supports, subtract pair intersections, and restore the triple intersection.

Signed intersections verify the logical gate

The phase $e^{i\pi W(v)/4}$ depends only on the remainder of $W(v)$ after division by eight. The notation $\equiv \pmod m$ means equal remainders after division by m. **Phase test:** each term, times its coefficient, must vanish mod 8 unless it is the logical triple.

$$W(g_1) \equiv 0 \pmod 8$$

individual term: coefficient 1,

Signed intersections verify the logical gate

The phase $e^{i\pi W(v)/4}$ depends only on the remainder of $W(v)$ after division by eight. The notation $\equiv \pmod m$ means equal remainders after division by m. **Phase test:** each term, times its coefficient, must vanish mod 8 unless it is the logical triple.

$$W(g_1) \equiv 0 \pmod 8$$

individual term: coefficient 1,

$$W(g_1 g_2) \equiv 0 \pmod 4$$

pair term: coefficient -2,

Signed intersections verify the logical gate

The phase $e^{i\pi W(v)/4}$ depends only on the remainder of $W(v)$ after division by eight. The notation $\equiv \pmod m$ means equal remainders after division by m. **Phase test:** each term, times its coefficient, must vanish mod 8 unless it is the logical triple.

$$W(g_1) \equiv 0 \pmod 8$$ individual term: coefficient 1,

$$W(g_1 g_2) \equiv 0 \pmod 4$$ pair term: coefficient -2,

$$W(g_1 g_2 g_3) \equiv \begin{cases} 1 \pmod 2, & \text{the three logical rows,} \\ 0 \pmod 2, & \text{every other triple.} \end{cases}$$

Higher intersections have coefficients divisible by eight. Only the logical triple remains, so for every $v \in S$,

$$W\big(K(a) \oplus v\big) \equiv 4a_1a_2a_3 \pmod 8, \qquad e^{i\pi \cdot 4a_1a_2a_3/4} = (-1)^{a_1a_2a_3}.$$

All 64 words at a fixed logical label receive this same phase: logical CCZ.

The resulting quantum code

48 physical qubits encode 3 logical qubits.

A stabilizer *check* tests an operation that fixes every encoded state. Quantum distance d_Q is the smallest number of sites in an undetected product of bit and phase flips that changes the encoded information.

My paper (Sources backup) establishes $d_Q = 3$ using the complete set of code checks. Distance three guarantees correction of an arbitrary error on one physical qubit. The notation $[\![n, k, d_Q]\!]$ lists physical qubits, logical qubits, and quantum distance:

$$[\![48, 3, 3]\!] \qquad \underbrace{26T + 22T^\dagger}_{\text{logical CCZ}}.$$

The phase test of slide 22 verifies the gate. The distance calculation establishes error protection.

Is 48 the smallest possible?

Reed–Muller codes and finite packing

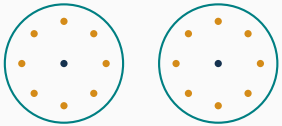
Back to the classical code of Part 1: it is the *Reed–Muller code* $\mathrm{RM}(1,3)$. In $\mathrm{RM}(\ell, m)$, ℓ bounds polynomial degree and m counts address bits.

A *linear code* is closed under XOR. Its parameters $[n, k, d_{\min}]$ specify stored bits, message bits, and minimum distance. Our code is $[8, 4, 4]$.

Reed–Muller codes and finite packing

Back to the classical code of Part 1: it is the *Reed–Muller code* $\mathrm{RM}(1,3)$. In $\mathrm{RM}(\ell, m)$, ℓ bounds polynomial degree and m counts address bits.

A *linear code* is closed under XOR. Its parameters $[n, k, d_{\min}]$ specify stored bits, message bits, and minimum distance. Our code is $[8, 4, 4]$.



Each center is a codeword.
Eight neighbors: its single-bit errors.

Each codeword has nine possible zero-or-one-error readouts. Unique correction (slide 9) makes these sets disjoint:

$$\underbrace{16}_{\text{codewords}} \underbrace{(1+8)}_{\text{readouts each}} = 144 \leq 256.$$

This packing count is the one-error *Hamming bound*. No one-error-correcting code of length 8 has more than $\lfloor 256/9 \rfloor = 28$ codewords.

Recovering missing data and approaching capacity

An *erasure* is a missing value whose position is known.



Every codeword has even weight (0, 4, or 8), so the missing bit is the XOR of the other seven: $1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 0$.

Recovering missing data and approaching capacity

An *erasure* is a missing value whose position is known.



Every codeword has even weight (0, 4, or 8), so the missing bit is the XOR of the other seven: $1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 0$.

Rate: message bits per transmitted bit. **Capacity:** the limiting rate compatible with vanishing recovery error as blocks grow, for a specified random channel.

When each bit is independently erased with a fixed probability, suitable growing Reed–Muller codes approach this limit under optimal decoding.

This is a long-block result about random losses, beyond our finite one-error guarantee.

Kudekar et al., Reed–Muller Codes Achieve Capacity on Erasure Channels, Theorem 27.

Predicting outputs, learning, and cryptography

With all eight addresses equally likely, parity predicts r on seven of eight inputs.

r	1	1	1	0	1	0	0	1
p	0	1	1	0	1	0	0	1

Address order $000, 001, \ldots, 111$: gold disagreement, teal agreement.

Predicting outputs, learning, and cryptography

With all eight addresses equally likely, parity predicts r on seven of eight inputs.

r	1	1	1	0	1	0	0	1
p	0	1	1	0	1	0	0	1

Address order $000, 001, \ldots, 111$: gold disagreement, teal agreement.

A *parity pattern* is an XOR of a subset of input bits, a linear function. **Fourier analysis** measures how often a function agrees with each parity pattern.

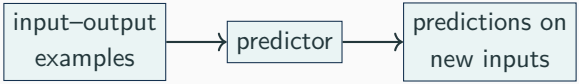
Predicting outputs, learning, and cryptography

With all eight addresses equally likely, parity predicts r on seven of eight inputs.

r	1	1	1	0	1	0	0	1
p	0	1	1	0	1	0	0	1

Address order $000, 001, \ldots, 111$: gold disagreement, teal agreement.

A *parity pattern* is an XOR of a subset of input bits, a linear function. **Fourier analysis** measures how often a function agrees with each parity pattern.



Statistical learning builds predictions from examples; suitable Boolean structure can help. **Cryptographic components** must resist prediction: r has degree three, yet parity predicts it on seven of eight inputs.

Contributions

Explicit construction. Forty-eight physical qubits encode three logical qubits and correct any one-qubit error.

Physical implementation. Twenty-six T and twenty-two $T^\dagger$ rotations implement logical CCZ, with no further gates.

Exact gate test. Signed row intersections verify one common phase throughout each encoded basis state.

Affine-function counts proved the classical distance. Boolean evaluation words and signed intersection counts specify the quantum encoding and verify its gate.

Where can I read more?

Affine equivalence changes input coordinates

Question from slide 8: Does changing binary coordinates change these counts or the code?

For m-bit inputs, let A be an invertible binary matrix and t an m-bit translation. Matrix addition uses XOR. Input-affine equivalence means

$$g(u) = f(Au \oplus t).$$

This bijection permutes table entries, preserving weight and polynomial degree (for nonzero functions). Applied to both words, it preserves Hamming distance.

For a direction v, the derivatives transform as

$$D_v g(u) = D_{Av} f(Au \oplus t).$$

Use the same address permutation for every codeword. The relabeled classical code has the same distances and correction guarantee.

▸ Return to slide 8

The eight-qubit quantum distance is two

Question from slide 16: Why does the eight-qubit code not correct every one-qubit error?

A *stabilizer check* tests an operation that fixes every encoded state. The eight-qubit code's checks detect every single-site bit or phase error. Write Z_u for a phase flip at physical address u.

$$Z_{000}Z_{100}|a\rangle_L = (-1)^{a_1}|a\rangle_L.$$

This two-site error passes every check. Both words within a pair receive the same phase, but the phase depends on the logical label.

Quantum distance counts the fewest sites in an undetected nontrivial logical bit/phase error. Here it is two: detection of one-site errors does not imply correction of an unknown one-site error.

Return to slide 16

The 48-qubit logical Boolean functions

Question from slide 19: What are the three logical functions?

Each block fixes w. The bits u_1, u_2, u_3, u_4 label positions inside it. Evaluate these expressions to obtain the three words K_1, K_2, K_3.

$$w = 10 \qquad \begin{aligned} K_1 &: u_1 \oplus u_3, \\ K_2 &: 1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4, \\ K_3 &: 1 \oplus u_1 \oplus u_1u_2 \oplus u_1u_4 \oplus u_3u_4. \end{aligned}$$

The eight combinations $K(a)$ select disjoint translates of the 64-word collection S.

Return to slide 19

The 48-qubit logical Boolean functions

Question from slide 19: What are the three logical functions?

Each block fixes w. The bits u_1, u_2, u_3, u_4 label positions inside it. Evaluate these expressions to obtain the three words K_1, K_2, K_3.

$$w = 01 \qquad \begin{aligned} K_1 &: 1 \oplus u_3, \\ K_2 &: 1 \oplus u_1 \oplus u_3 \oplus u_1u_3 \oplus u_1u_4, \\ K_3 &: u_1 \oplus u_1u_2 \oplus u_2u_3 \oplus u_2u_4 \oplus u_3u_4. \end{aligned}$$

The eight combinations $K(a)$ select disjoint translates of the 64-word collection S.

Return to slide 19

The 48-qubit logical Boolean functions

Question from slide 19: What are the three logical functions?

Each block fixes w. The bits u_1, u_2, u_3, u_4 label positions inside it. Evaluate these expressions to obtain the three words K_1, K_2, K_3.

$$w = 11 \qquad \begin{aligned} K_1 &: u_4, \\ K_2 &: u_3 \oplus u_4 \oplus u_1u_3 \oplus u_1u_4, \\ K_3 &: u_1 \oplus u_3 \oplus u_1u_4 \oplus u_2u_3 \oplus u_2u_4. \end{aligned}$$

The eight combinations $K(a)$ select disjoint translates of the 64-word collection S.

Return to slide 19

The 48-qubit rotation-sign polynomial

Question from slide 20: How is the rotation pattern specified?

For the four within-block bits u and two block bits w, define

$$Q_1(u) = u_1 \oplus u_2 \oplus u_3 \oplus u_1u_3 \oplus u_2u_3 \oplus u_2u_4,$$
$$Q_2(u) = u_3 \oplus u_4 \oplus u_1u_2 \oplus u_1u_4 \oplus u_3u_4.$$

The bit-valued sign function is

$$\gamma(u, w) = u_3 \oplus w_1Q_1(u) \oplus w_2Q_2(u).$$

Value zero selects T; value one selects $T^\dagger$. On blocks $10, 01, 11$, the numbers of ones are $6, 6, 10$, giving 22 inverse and 26 forward rotations.

Return to slide 20

The scope of the length exclusion

Question from slide 23: Is 48 the smallest possible length?

Hypotheses: *CSS codes* (separate bit-flip and phase-flip checks), three logical qubits, and $d_Z \geq 3$ (no undetected phase-type logical error on one or two sites).

Also require the *Campbell–Howard conditions*: nine even/odd requirements on rows and their pair/triple intersections.

physical length	conclusion under these hypotheses
$n \leq 38$	excluded at every number of independent stabilizer rows
$39 \leq n \leq 46$	existence remains open
$n = 47$	one specified code admits no $T/T^\dagger$-only CCZ pattern
$n = 48$	the displayed construction implements that pattern

The construction does not establish global minimality.

▸ Return to slide 23

Sources

Question from slide 27: Where can I read more?

Quantum construction and bounds: the author's paper, *A 48-qubit native CCZ code from projective geometry and a length lower bound*.

Classical coding: MacWilliams–Sloane, *The Theory of Error-Correcting Codes*; Huffman–Pless, *Fundamentals of Error-Correcting Codes*.

Boolean functions: Carlet, *Boolean Functions for Cryptography and Coding Theory*; O'Donnell, *Analysis of Boolean Functions*, Chapters 1–3 (Fourier analysis, input influence, learning).

Erasure capacity: Kudekar et al., *Reed–Muller Codes Achieve Capacity on Erasure Channels*, Theorem 27.

Memory context: IBM, *Troubleshooting Memory issues*. The eight-bit encoder is a teaching example.

Return to slide 27